Graph-View Core Specification

Renderer-independent graph data, selection, complexity, and errors

mdstats

Contents

1 Purpose and status

This document is the normative specification for the renderer-independent graph view implemented in `mdstats.plotting.graph_view` and for the shared exception hierarchy in `mdstats.plotting.graph_errors`.

It owns graph-view data validation, immutable metadata, stable scientific keys, focus selection, deterministic filters, omission accounting, and rendering complexity policies. It does not own style resolution, layout, Matplotlib artists, or scientific adapters.

2 Motive

Scientific graph modules have different node and edge meanings, but renderers need a uniform data contract. `DecoratedGraphView` supplies that contract without replacing or mutating the authoritative scientific result.

The dense endpoint representation is optimized for layout and drawing, while stable keys preserve scientific identity:

[stable key dense array position.]

3 Normative ownership

This specification owns:

- `GraphKey`, `AttributeScalar`, and `AttributeColumn` contracts;
- `DecoratedGraphView`;
- `GraphFocus`, `AttributeSelection`, and `GraphFilter`;
- `GraphComplexityPolicy` and `GraphComplexityReport`;
- the internal `PreparedGraphView` selection contract;
- the public graph-visualization exception hierarchy.

Styles and renderers consume these objects but must not redefine them.

4 AI context summary

- The source scientific graph remains authoritative.
- Keys are stable identities; endpoint arrays contain dense positions only.
- All public graph-view objects are deeply immutable where practical.
- Focus is applied before explicit filters.
- Edges survive only when both endpoints survive node selection.
- Omitted keys are recorded.
- No silent graph sampling is allowed.
- Nonzero image shifts require positions, a cell, and PBC flags.
- Display selection never changes the source view.

5 Mathematical conventions

5.1 Cartesian and cell convention

Lattice vectors are rows of the cell matrix

$$H = \begin{pmatrix} \mathbf{a}^\mathsf{T} \\ \mathbf{b}^\mathsf{T} \\ \mathbf{c}^\mathsf{T} \end{pmatrix}.$$

For fractional coordinate $\mathbf{s}_i$, the Cartesian coordinate is

$$\mathbf{r}_i = \mathbf{s}_i H.$$

`node_positions_3d` always stores Cartesian positions.

5.2 Periodic edge geometry

For an edge from node i to an image of node j with integer shift $\mathbf{m}_{ij}$, the physical edge vector is

$$\mathbf{d}_{ij} = \mathbf{r}_j + \mathbf{m}_{ij} H - \mathbf{r}_i.$$

A two-dimensional physical projection $P \in \mathbb{R}^{2\times 3}$ gives

$$\mathbf{x}_i = P\mathbf{r}_i,$$

and

$$\mathbf{x}_j^{\text{image}} = P\left(\mathbf{r}_j + \mathbf{m}_{ij} H\right).$$

5.3 Schematic coordinates

Schematic coordinates $\mathbf{q}_i \in \mathbb{R}^2$ are generated from graph connectivity rather than physical position. They must be labeled in render metadata as schematic:

$$\mathbf{q}_i \not\equiv P\mathbf{r}_i.$$

A schematic graph must not be described as a geometric atomistic structure.

6 Common public type contracts

6.1 Graph keys

```
from collections.abc import Hashable
from typing import TypeAlias

GraphKey: TypeAlias = Hashable
```

A graph key must:

- be hashable;
- have deterministic equality during one process and across serialization when persistence is intended;
- be immutable by contract;
- not contain a floating-point NaN;
- remain unique within its node-key or edge-key collection.

Recommended keys include:

```
atomic node         integer atom index
atomic edge         AtomicEdgeKey or a stable tuple
framework node      retained atom index
framework edge      stable projected-edge ID
ring node           stable ring ID
site or cage node   stable site/cage ID
```

Custom object keys are permitted, but portable serialization is guaranteed only for keys whose values have a stable external representation.

6.2 Attribute values and columns

```
AttributeScalar = (
    None | bool | int | float | str |
    tuple["AttributeScalar", ...]
)

AttributeColumn = (
    NDArray[Any] |
    tuple[AttributeScalar, ...]
)
```

Attribute columns must be one-dimensional and must have the same length as the corresponding node or edge collection.

Attribute names must:

- be nonempty strings;
- be unique within the node or edge attribute mapping;
- not begin with `_mdstats_`, which is reserved for internal use.

Lists, dictionaries, mutable sets, and mutable custom objects are not valid attribute values. Missing categorical data should use `None`. Numeric metadata should be finite; missing numeric data should use `None` rather than NaN or infinity.

7 DecoratedGraphView

7.1 Public definition

```
@dataclass(frozen=True, slots=True)
class DecoratedGraphView:
    node_keys: tuple[GraphKey, ...]
    edge_keys: tuple[GraphKey, ...]
    edge_endpoints: NDArray[np.int64]

    node_positions_3d: NDArray[np.float64] | None = None
    edge_image_shifts: NDArray[np.int64] | None = None

    cell: NDArray[np.float64] | None = None
    pbc: NDArray[np.bool_] | None = None

    node_attributes: Mapping[str, AttributeColumn] = field(
        default_factory=dict
    )
    edge_attributes: Mapping[str, AttributeColumn] = field(
        default_factory=dict
    )

    directed: bool = False
```

```
    multigraph: bool = False
    metadata: Mapping[str, Any] = field(default_factory=dict)
```

7.2 Shape invariants

Let

$$N = |V|, \qquad M = |E|.$$

The following shapes are required:

```
node_keys              length N
edge_keys              length M
edge_endpoints         (M, 2)
node_positions_3d      (N, 3), when present
edge_image_shifts      (M, 3), when present
cell                    (3, 3), when present
pbc                     (3,), when present
node attribute column  length N
edge attribute column  length M
```

7.3 Validation rules

The constructor must validate that:

1. node keys are unique;
2. edge keys are unique;
3. endpoint positions are integers in `[0, N)`;
4. coordinate and cell arrays are finite;
5. the cell is nonsingular when present;
6. periodic shifts are integers;
7. shifts are zero on nonperiodic axes;
8. nonzero shifts require `cell`, `pbc`, and physical node coordinates;
9. every attribute column has the correct length;
10. metadata and attributes satisfy the immutability contract;
11. an undirected non-multigraph has no duplicate unordered endpoint pair;
12. a directed non-multigraph has no duplicate ordered endpoint pair.

An empty graph is valid. A graph with nodes and no edges is valid.

Self-loops and parallel edges may be represented when `multigraph=True`, but the initial Matplotlib renderer may reject self-loops and may require an explicit overlap option for parallel edges. This preserves future data compatibility without claiming full first-release rendering support.

7.4 Deep immutability

Construction must defensively copy arrays, make them C-contiguous, and set them read-only. Attribute mappings and metadata mappings must be copied and exposed as read-only mappings. Nested metadata values must be recursively frozen or validated as immutable.

A caller modifying an input array or dictionary after construction must not change the view.

7.5 Stable keys versus endpoints

`edge_endpoints[e]` contains positions in `node_keys`; it does not contain scientific node IDs directly.

For example:

```
node_keys = (12, 37, 81)
edge_endpoints[0] = (0, 2)
```

means that edge 0 connects scientific nodes 12 and 81.

This separation is required for efficient array processing and stable external identity.

7.6 View methods

The first implementation should provide:

```
@property
def n_nodes(self) -> int: ...

@property
def n_edges(self) -> int: ...

def node_position(self, key: GraphKey) -> int: ...

def edge_position(self, key: GraphKey) -> int: ...

def to_dict(self) -> dict[str, Any]: ...
```

`to_dict()` is intended for diagnostics and future serialization. It must not serialize Matplotlib objects or arbitrary nonportable custom keys silently.

8 Private `PreparedGraphView`

Focus, filtering, and later periodic replication produce a display-only graph. This must be represented privately rather than by mutating `DecoratedGraphView`.

A conceptual private structure is:

```
@dataclass(frozen=True, slots=True)
class PreparedGraphView:
    source_view: DecoratedGraphView
    node_source_positions: NDArray[np.int64]
    edge_source_positions: NDArray[np.int64]
    edge_endpoints: NDArray[np.int64]
    node_positions_3d: NDArray[np.float64] | None
    edge_image_shifts: NDArray[np.int64] | None
    selection_metadata: Mapping[str, Any]
```

Future periodic replicas may add node-instance translations and canonical source keys without changing the public view.

9 Focus and filtering

9.1 `GraphFocus`

```
@dataclass(frozen=True, slots=True)
class GraphFocus:
    center_node_keys: tuple[GraphKey, ...]
    hop_radius: int = 1
    direction: Literal["both", "out", "in"] = "both"
```

Rules:

- `center_node_keys` must be nonempty and unique;
- every center key must exist in the input view;
- `hop_radius` must be a nonnegative integer;
- for undirected graphs, `direction` is ignored;
- multiple centers produce the union of their neighborhoods;

- the selected graph is the node-induced subgraph of the selected nodes.

Focus is computed on the original scientific view before metadata filtering. This ensures that graph distance has one stable meaning even when a later filter hides intermediate node categories.

9.2 AttributeSelection

```
@dataclass(frozen=True, slots=True)
class AttributeSelection:
    attribute: str
    include_values: tuple[AttributeScalar, ...] | None = None
    exclude_values: tuple[AttributeScalar, ...] = ()
```

Exact equality is used. When `include_values` is not `None`, the value must be in the include set. Exclusions are applied afterward and therefore win.

9.3 GraphFilter

```
@dataclass(frozen=True, slots=True)
class GraphFilter:
    include_node_keys: tuple[GraphKey, ...] | None = None
    exclude_node_keys: tuple[GraphKey, ...] = ()

    include_edge_keys: tuple[GraphKey, ...] | None = None
    exclude_edge_keys: tuple[GraphKey, ...] = ()

    node_attribute_selections: tuple[
        AttributeSelection, ...
    ] = ()
    edge_attribute_selections: tuple[
        AttributeSelection, ...
    ] = ()

    keep_isolated_nodes: bool = True
```

Filter semantics:

1. begin with the focused graph or the entire view;
2. intersect all node inclusion constraints;
3. remove all explicitly excluded nodes;
4. retain only edges whose two endpoints remain;
5. intersect all edge inclusion constraints;
6. remove explicitly excluded edges;
7. optionally remove nodes isolated by filtering.

Unknown keys or attributes are errors, not ignored requests.

Callable predicates are deferred because they are difficult to serialize, reproduce, and inspect automatically.

10 Complexity safeguards

10.1 GraphComplexityPolicy

```
@dataclass(frozen=True, slots=True)
class GraphComplexityPolicy:
    max_nodes: int = 1500
    max_edges: int = 3000
    max_labels: int = 100
    max_gradient_segments: int = 20000
    overflow: Literal[
```

```
        "error",
        "require_focus",
        "warn_and_render",
    ] = "require_focus"
```

All limits must be positive integers.

`require_focus` means that the graph is processed through focus and filtering, then must fit within all applicable limits. If it still exceeds a limit, rendering fails with a targeted message containing the counts and suggested controls.

The renderer must never silently sample nodes, edges, labels, or gradient segments.

10.2 `GraphComplexityReport`

```
@dataclass(frozen=True, slots=True)
class GraphComplexityReport:
    input_nodes: int
    input_edges: int
    selected_nodes: int
    selected_edges: int
    requested_labels: int
    estimated_gradient_segments: int
    exceeded_limits: tuple[str, ...]
```

The report is included in every render result, including successful renders.

11 Shared exception hierarchy

The following public exceptions are defined in `graph_errors.py`:

```
class GraphVisualizationError(Exception): ...
class GraphViewValidationError(GraphVisualizationError): ...
class GraphFilterError(GraphVisualizationError): ...
class GraphStyleError(GraphVisualizationError): ...
class GraphLayoutError(GraphVisualizationError): ...
class GraphComplexityError(GraphVisualizationError): ...
class GraphAdapterError(GraphVisualizationError): ...
class GraphOptionalDependencyError(GraphVisualizationError): ...
class GraphUnsupportedFeatureError(GraphVisualizationError): ...
```

The core specification owns the hierarchy. Module specifications state which subclasses their APIs raise.

12 Selection algorithm

`prepare_graph_view()` is an internal helper. It must apply operations in this order:

1. resolve `GraphFocus` by graph distance;
2. apply explicit node-key inclusion and exclusion;
3. apply node-attribute selections;
4. retain only edges whose two endpoints remain;
5. apply explicit edge-key inclusion and exclusion;
6. apply edge-attribute selections;
7. optionally remove isolated nodes;
8. sort surviving source positions and reindex dense endpoints;
9. record omitted node and edge keys.

For directed graphs, focus may follow incoming, outgoing, or both directions. For undirected graphs the direction option has no effect on adjacency symmetry.

13 Serialization and provenance

`DecoratedGraphView.to_dict()` provides a portable inspection representation. It is not a structural graph digest and must not be used as scientific identity.

Metadata must be recursively copied and frozen. Supported values are finite scalar values, tuples, arrays, mappings with nonempty string keys, frozen sets, and immutable hashable dataclass values such as `AtomicEdgeKey`.

14 Input constraints and edge cases

Reject:

- duplicate or unhashable node or edge keys;
- NaN keys;
- endpoint positions outside the node array;
- duplicate endpoint pairs in a non-multigraph;
- malformed or nonfinite coordinate arrays;
- singular cells;
- image shifts on nonperiodic axes;
- metadata columns of incorrect length;
- mutable or nonportable attribute values;
- unknown focus, filter, or attribute keys.

Empty graphs are valid. Isolated nodes are valid. Disconnected graphs are valid. A disconnected graph is not inherently erroneous because it may represent molecules, multiple slabs, fragments, or a deliberately selected subgraph.

15 Testing requirements

Tests must cover:

- immutable defensive copies of arrays, mappings, and metadata;
- stable keys versus dense endpoint positions;
- empty, isolated, disconnected, directed, and multigraph cases;
- periodic-shift validation;
- exact filter order and omitted-key accounting;
- directed focus semantics;
- unknown-key and unknown-attribute failures;
- complexity-limit reporting with no silent sampling;
- serialization of standard keys and immutable scientific keys.

16 Public exports

The public exports owned here are:

```
AttributeSelection
DecoratedGraphView
GraphComplexityPolicy
GraphComplexityReport
GraphFilter
GraphFocus
```

`GraphKey`, `AttributeScalar`, and `AttributeColumn` are public module-level typing contracts. `PreparedGraphView` and `prepare_graph_view()` are internal implementation contracts and are not exported from `mdstats.plotting`.

17 Future compatibility

The future periodic-display module may consume `DecoratedGraphView` and produce a separate prepared display graph with replica mappings. It must not overload `PreparedGraphView`, whose current purpose is selection and source-position tracking. Future 3-D, framework, ring, site, and cage adapters must continue to emit stable keys and columnar metadata under this contract.